Athel — 01 Master PRD

Spec-Driven Agentic Development Framework · Nov 1, 2026 · @Vaibhav Shukla

Document map

Athel is planned across eight documents: two master documents set the whole picture, five module documents each hold a PRD part and a TRD part, and one implementation plan sequences the build.

#	Document	Holds	Review focus
01	Master PRD (this doc)	Problem, vision, features, journey, metrics, phases	Is this the right product?
02	Master TRD	Architecture, plugin layout, state model, workflow engine, cross-cutting rules	Is this the right system?
03	Discovery and Integrations	Scout agent, live discovery, verified knowledge cache, every tool integration	Can it really understand the system?
04	Safety, Guardrails and Git	Permission tiers, hooks, audit, secrets, git policy, multi-repo safety	Can it ever cause harm?
05	Spec, Blast Radius and Planning	PRD intake, specs, decision cards, impact analysis, change plans	Will it build the right thing?
06	Build Loop, Review and Quality	Role agents, iterative review, gates, code quality standards	Will it build the thing right?
07	Verification, UAT and Learning	Staging checks, side findings, UAT rounds, lessons	Does it get better over time?
08	Implementation Plan	Milestones, build order, test strategy for Athel itself, exit criteria	Can we ship it step by step?

Open the others: SDADF — 02 Master TRD · SDADF — 03 Discovery and Integrations · SDADF — 04 Safety, Guardrails and Git · SDADF — 05 Spec, Blast Radius and Planning · Doc · Doc · Doc

How to review: read 01 and 02 first, since every module document depends on their terms. Requirement IDs (for example `FR-DISC-004`) are stable, so comments can point at one line. Anything marked **Open question** needs a decision from you before implementation starts.

Terms used everywhere:

- **Change**: one unit of work, from a PRD or request to merged PRs. Has an ID like `CHG-042`.
- **Change set**: all branches and PRs across repos that belong to one change.
- **Fact**: a stored piece of system knowledge with its source and a way to re-verify it.
- **Gate**: an automated check that must pass before work moves forward. Gates block; they never just warn.
- **Decision card**: a structured question to a human with options and a recommendation.
- **Side finding**: an existing problem noticed while working, outside the change's scope.
- **Blast radius**: everything a change can affect, inside and outside the repo being edited.

Problem statement

Today, getting Claude Code to deliver a real feature means writing one huge prompt, and that prompt fails at exactly the parts of software development that matter most in a large system.

Why the big prompt fails

- It is written once and goes stale as the work moves; there is no durable spec, plan or state.
- It mixes rules, context, steps and goals, so Claude cannot tell what is a hard rule and what is a hint.
- It has no memory: feedback from one round is lost by the next session.
- It has no enforcement: "run the tests" is a request, not a gate.

The true gaps of software development that no current tool closes

Gap	What happens today	What Athel must do
System understanding	Agents see one repo; the other hundred services are invisible	Discover owners, contracts, callers and real traffic across services, live
Multi-service change	A change that spans five services is done by hand, one repo at a time	Plan and execute a linked change set in a safe order

Gap	What happens today	What Athel must do
Blast radius	Impact is guessed; breaking consumers is found in production	Report impact before coding and before every PR, with real traffic numbers
Git discipline	Agents create messy branches and can collide with teammates	Follow a git policy, detect other people's work, never touch it
Asking well	Agents either guess silently or ask vague questions	Ask only when needed, with options and a recommendation
Existing issues	Problems noticed along the way are ignored or silently changed	Log them, rank them, ask before fixing, fix in separate PRs
Production reality	Code is read; dashboards, logs and infra are not	Use observability and infra tools to see how the system really behaves
Learning	The same correction is given again and again	Turn UAT feedback into rules that apply to every future run

The existing spec-driven tools (GitHub Spec Kit, Kiro, BMAD-METHOD, OpenSpec) structure work inside one repo. None of them treats the whole running system as its context, which is the gap Athel targets.

Vision, principles and non-goals

Vision: a human gives Athel a PRD and later gives UAT feedback; in between, Athel works like a careful senior engineering team that understands the whole running system, changes it safely across services, and gets better with every round.

Athel is a Claude Code plugin. Claude Code is the reasoning and coding engine; Athel owns the method, the rules, the state and the safety.

Principles (every design decision in docs 02 to 08 must trace back to one of these)

1. **Live tools are the truth, stored knowledge is a hint.** Any stored fact is re-verified against its source before it drives an action.
2. **Safe before smart.** Guardrails ship before capabilities. Anything irreversible needs a human.
3. **Gates block, they never just warn.** A failed check stops the flow until fixed or explicitly waived by a human.
4. **Read freely, write narrowly.** Reading code, logs and dashboards is broad; writing is limited to the change's own branches and approved environments.

5. **Never touch other people's work.** No edits to branches, PRs or files others are actively changing.
6. **Ask rarely, ask well.** Decide alone when the answer is reversible and covered by rules; otherwise ask with options and a recommendation.
7. **Everything is a file.** Specs, plans, decisions, findings and lessons live on disk in readable form, so any session can resume and any human can audit.
8. **Small, reviewable steps.** Small commits, small PRs, one concern each.
9. **Every action is explainable.** For any edit or command, Athel can show why it happened and which spec line it serves.
10. **Humans own merges and production.** Athel opens PRs; humans merge and deploy. Athel never deploys to any environment, staging included, and never changes infrastructure.

Non-goals for v1

- Supporting coding engines other than Claude Code.
- Merging PRs, deploying to any environment (staging included), or changing infrastructure.
- Replacing CI/CD, code review by humans, or incident response.
- A web UI. The interface is Claude Code itself plus the files in `.athel/`.
- Maintaining a complete, permanent map of all services. Knowledge is discovered on demand and cached as hints.

Users, personas and primary use cases

The first user is a single developer driving Athel from Claude Code; teams and the wider org come later.

Persona	Goal	What they give Athel	What they get back
Feature owner (PM or dev)	Ship a feature from a PRD	PRD, UAT feedback	Linked PRs, blast radius, test and security reports
Engineer on an existing service	Change one service safely	A request or ticket	A small PR plus impact on other services
Reviewer	Approve with confidence	Review comments	PRs with specs, decisions and evidence attached
Tech lead or platform owner	Keep the system healthy	Policies (git, tools, gates)	Enforced rules, audit log, side findings

Persona	Goal	What they give Athel	What they get back
On-call engineer (later)	Understand an issue fast	An alert or symptom	An investigation report from logs, metrics and code

Primary use cases

ID	Use case	First supported in
UC-01	Ask a question about the system ("who owns customer address?", "what calls this endpoint?") and get a verified answer	Phase 1
UC-02	Get a blast radius report for a proposed change before writing code	Phase 1
UC-03	Turn a PRD into specs, decisions and a plan for one repo	Phase 2
UC-04	Build a feature in one existing repo, with tests, review and gates	Phase 3
UC-05	Build a new project from scratch from a PRD	Phase 3
UC-06	Execute a change across several services as a linked change set	Phase 4
UC-07	Get a deploy-and-test guide, then verify the developer's staging deploy using dashboards, logs and staging data	Phase 5
UC-08	Run UAT rounds where feedback becomes changes and lessons	Phase 5
UC-09	Review side findings and approve small, separate fixes	Phase 3
UC-10	Resume any change in a new session with no lost context	Phase 2

Feature catalogue

Athel has 24 features in 7 groups; P0 is required for the first usable release, P1 for the full vision, P2 is later.

ID	Feature	What it does	Priority	Detailed in
F-01	Guardrail hooks	Block forbidden actions (prod writes, secret reads, push to main, force push)	P0	04
F-02	Audit log	Record every external tool call and every file write with the reason	P0	04
F-03	Tool policy	Per tool and per environment: read, write with approval, or forbidden	P0	04
F-04	Scout agent	Read-only investigator that explores code, infra, observability, tickets and chat	P0	03
F-05	Verified knowledge cache	Store facts with source and re-check recipe; verify before use	P0	03
F-06	Tool integrations	Adapters and playbooks for GitHub, kubectl, gcloud, Praxis (Facets), Grafana, Coralogix, Jira, Linear, Slack, Teams and more	P0 (core set)	03
F-07	Blast radius report	Direct, downstream, data, infra and team impact with real traffic and a risk level	P0	05
F-08	PRD intake and spec	PRD to requirements, design and acceptance criteria, with gaps found	P0	05
F-09	Decision cards	Structured questions with options, recommendation and default	P0	05
F-10	Change plan	Ordered task graph across repos with safe sequencing	P0	05
F-11	Git preflight	Fetch, detect others' branches, PRs and recent edits, cut branches by policy	P0	04
F-12	Build loop	Per task: implement, test, security, review, fix, until gates pass	P0	06
F-13	Quality gates	Build, tests, lint, types, coverage, security scan, contract checks	P0	06

ID	Feature	What it does	Priority	Detailed in
F-14	Iterative review	Independent reviewer agents, findings tracked to closure, re-review after fixes	P0	06
F-15	Session resume	Any change can continue in a new session from files alone	P0	02
F-16	Side findings	Log existing issues, rank by risk and effort, ask, fix in separate PRs	P1	07
F-17	Multi-repo change sets	Linked branches and PRs across repos with one change ID and merge order	P1	04, 05
F-18	Staging verification	Deploy-and-test guide for the developer; after the developer deploys, read-only checks of dashboards, logs, error rates and staging data against a baseline	P1	07
F-19	UAT rounds	Structured feedback becomes spec changes and the next build round	P1	07
F-20	Learning	Feedback and review patterns become rules applied to future runs	P1	07
F-21	Brownfield onboarding	Derive specs and conventions from an existing repo before changing it	P1	05
F-22	Parallel execution	Independent tasks run in separate git worktrees at once	P2	06
F-23	Athel evals	A suite of sample PRDs that measures Athel quality release over release	P1	08
F-24	Org-wide policies	Shared rules, lessons and tool policies across teams	P2	02

End-to-end journey for one change

A human touches the flow at four points only: giving the PRD, answering decision cards, approving the plan, and UAT; everything else runs on its own and stops at a gate when something fails. The state machine behind this is in doc 02.

Example: "Let customers update their phone number from the app; all services must see the new number."

1. **Intake** (`/athel:start prd.md`): Athel creates `CHG-042` , copies the PRD, and checks it for missing parts (no acceptance criteria, unclear actors, conflicting requirements).
2. **Investigate**: the scout finds that user-service owns the phone number, notification-service caches it, and crm-sync consumes the `UserUpdated` event. It confirms this from code, Kubernetes services, Praxis (Facets) and Grafana traffic, and notes that a teammate has an open PR touching user-service's profile handler.
3. **Spec**: requirements, design and acceptance criteria per service, each traced to a PRD line.
4. **Blast radius**: three services change, two read-only consumers are affected, one migration touches 2.1M rows; risk HIGH because the API contract changes.
5. **Decision cards**: two real questions only, for example "Validate numbers with E.164 only, or also accept local formats? Recommended: E.164 with a local-format parser at the edge."
6. **Plan approval (human)**: ordered tasks per repo. Provider first, backward-compatible; consumers next; old field removed in a later change.
7. **Git preflight**: per repo, fetch latest, check the teammate's PR, cut `athel/CHG-042-phone-update` from the policy's base branch.
8. **Build loop**: per task, developer agent implements; QA writes and runs tests in the repo's existing test style; security reviews; reviewer reviews; findings are fixed and re-reviewed until every gate passes.
9. **PRs**: linked PRs open with the spec, blast radius, decisions, test evidence and side findings attached. Merge order is stated. Humans review and merge.
10. **Deploy guide and verify**: Athel never deploys, not even to staging. It checks what a merge will trigger (for example, an automatic staging deploy through the pipeline or Facets), writes a plain guide telling the developer how to deploy and what to test on staging and later on prod, and after the developer deploys, verifies staging read-only: acceptance checks, error rate and latency against the pre-change baseline, and staging data if a read-only database user is configured.
11. **UAT (human)**: feedback in plain words, for example "the success message should show the masked number".
12. **Learn and repeat**: feedback becomes a spec change for round 2, and, if general, a lesson such as "always mask phone numbers in user-facing text" that applies to every future change once you confirm it.

If any step fails (a gate, a tool, a verification), the change moves to a blocked state with the reason on disk, and Athel either retries within limits or raises a decision card.

Success metrics and acceptance criteria

Athel succeeds when it is safe first, then accurate, then useful; the targets below are starting proposals to confirm after the first ten real changes.

Area	Metric	Target	How measured
Safety	Guardrail violations reaching a real system	0, always	Audit log plus hook block records
Safety	Edits to branches or files owned by others	0	Git preflight records vs PR history
Accuracy	Blast radius recall (affected services found / actually affected)	95% or higher	Post-merge review of incidents and consumer changes
Accuracy	Stale facts used without re-verification	0	Cache access log
Quality	PRs passing all gates on first human review	80% or higher	PR outcomes
Quality	Escaped defects found in UAT per change	Falls each month	UAT records
Asking	Decision cards per change	5 or fewer, median 2	Change records
Asking	Cards a human marks "should not have asked"	Under 10%	Card feedback field
Learning	Repeat of a UAT correction already captured as a lesson	Under 5%	Lesson match on new feedback
Speed	Time from approved plan to PRs for a 3-service change	Under 1 working day	Change timestamps
Resume	Changes resumed in a new session without human re-explaining	100%	Session start records

Release acceptance criteria (all must hold)

- Every P0 feature has passing automated tests and a passing run on at least three eval PRDs (doc 08).
- A red-team test set of forbidden actions is blocked 100% by hooks, including attempts phrased inside tool output.

- Every requirement in docs 03 to 07 has at least one test or a written waiver.
- A full change can be stopped at any step and resumed in a fresh session with no loss.

Release phases, risks, assumptions and decisions

Athel ships in six phases, each usable on its own; the full milestone plan with exit criteria is in doc 08.

Phase	Delivers	Features	Why this order
0. Foundation	Plugin skeleton, guardrails, audit, tool policy, state files	F-01, F-02, F-03, F-15	Safety must exist before any capability
1. Understand	Scout, verified cache, core integrations, blast radius (read-only)	F-04 to F-07	Useful daily with zero write risk; proves live discovery
2. Specify	PRD intake, specs, decision cards, plans, brownfield onboarding	F-08 to F-10, F-21	Right thing before building things
3. Build (one repo)	Git preflight, build loop, gates, iterative review, side findings	F-11 to F-14, F-16	Prove quality in a single repo first
4. Multi-service	Linked change sets across repos	F-17	Highest value and highest risk; needs 0 to 3 solid
5. Verify and learn	Staging verification, UAT rounds, learning, evals	F-18 to F-20, F-23	Closes the loop
Later	Parallel worktrees, org-wide policies	F-22, F-24	Scale after correctness

Top risks

Risk	Impact	Mitigation
An agent runs a destructive command through a logged-in tool	Outage or data loss	Hook-enforced policy, prod read-only, scoped accounts, red-team tests (doc 04)
Prompt injection via logs, tickets, PR text or chat	Agent obeys an attacker	Tool output treated as data; write actions need policy plus intent from the change plan (doc 04)

Risk	Impact	Mitigation
Wrong system picture from stale or partial data	Missed consumers, broken services	Verify-before-use, multiple sources per fact, confidence levels (doc 03)
Context window exhaustion on big changes	Lost reasoning, repeated work	State on disk, subagents with isolated context, summaries per step (doc 02)
Claude Code features change	Plugin breaks	Thin adapter layer, version pinning, compatibility tests (doc 02)
Cost and time per change too high	Nobody uses it	Budgets per change, cache hits, model choice per role (doc 02)
Over-asking or under-asking	Annoyance or wrong decisions	Explicit ask rules plus measured card feedback (doc 05)

Assumptions

- The developer's machine has Claude Code plus authenticated CLIs and MCP servers for the tools in use.
- Staging environments exist, and developers deploy to them through the existing pipeline or Facets.
- Repos have a test command that can run locally or in CI.

Decisions made

- Git host and CI for the pilot: GitHub and GitHub Actions.
- Praxis (Facets read CLI) joins the first integration set; Raptor (Facets write CLI) is registered but every call is blocked (doc 03).
- Athel is never allowed to deploy to staging itself or anywhere else; it writes a deploy-and-test guide and verifies after the developer deploys (doc 07).
- Pilot: Athel must handle both a new project and an existing service from the first build milestone.